DSL for Satellite Specifications

To support specification-driven testing of CubeSat systems, we introduce a DSL tailored to describe the structure, behavior, and interactions of CubeSat subsystems. This DSL serves as the specification language for the framework, providing a common foundation for test generation and validation.

1 Syntax

Specification	$\mathcal{S}$	:=	$\mathbb{M}^*\ \mathbb{R}^*\ \mathbb{I}^*\ \mathbb{P}^*\ \mathbb{O}^*$
Import	$\mathbb{M}$	:=	`#import` $\mathbf{s}$
Resource	$\mathbb{R}$	:=	`resource` c `:` τ
Interface	$\mathbb{I}$	:=	`interface` x `:` c `{` $(v : c)^*$ `}`
Protocol	$\mathbb{P}$	:=	`protocol` c `{` a^* `}`
Object	$\mathbb{O}$	:=	(`object` | `objectC`) c `{` d^* (`algorithm init (`x `:` `unit`) `:` `unit` `{` e `}`)$^?$ $\mathbb{C}^*$ w `}`
Algorithm	a	:=	`algorithm` x`(`x `:` τ`)` `:` τ `{` e `}`
Data	d	:=	`data` x `:` τ
Component	$\mathbb{C}$	:=	`component` c `:` $\tau_{\mathbb{C}}$ `{` a^* p^* `}`
Pipe	p	:=	`pipe` x(`@` c^+)$^?$ `:` τ_{arrow} `{` $\mathbf{p}$ `}`
PipeExpr	$\mathbf{p}$	:=	$\mathbf{p}_b$(`@` ρ^+)$^?$ | $\mathbf{p}$ `--` $\mathbf{p}$
PipeBasic	$\mathbf{p}_b$	:=	$\mathbf{p}_l$ | $\mathbf{p}_f$`(`$\mathbf{p}_c{}^+$`)` | `matchthis` (`|` e_p `=>` $\mathbf{p}$)$^+$ `end`
PipeName	$\mathbf{p}_l$	:=	`sender` | `receiver` | `ignorer` | $l \notin$ {`sender`, `receiver`, `ignorer`}
PipeFunc	$\mathbf{p}_f$	:=	`constant` | `sleeper` | `syncComm` | `syncCommVal` | `looper`
PipeDef	$\mathbf{p}_c$	:=	$\mathbf{p}$ | `$`e
Worker	w	:=	`workers [` $\mathbf{w}^*$ `]`
WorkerDef	$\mathbf{w}$	:=	`Listen` $c.x$ `on` x`[`v`]`
		|	`Trigger` $c.x$ `every` δ `s`
		|	`Init` $c.x$
PortExpr	ρ	:=	`_` | c | x`[`v`]`
PeriodExpr	δ	:=	$\mathbf{n}$ | `(`e`)`
BasicExpr	e	:=	v | x | `(`e`)` | `~`e | e *bop* e
		|	x `=` e `;` e | c`(`e^*`)`
		|	`match` e `with` (`|` e_p `=>` e)$^+$ `end` | `call` l`(`e^*`)`
BinaryOp	*bop*	:=	`+` | `-` | `*` | `/` | `&` | `|` | `^` | `<<` | `>>` | `>>>`
ExprPattern	e_p	:=	`_` | x | v | c`(`$e_p{}^*$`)`
Name	l	:=	x | $c.x$
Value	v	:=	$\mathbf{n}$ | $\mathbf{h}$ | $\mathbf{b}$ | $\mathbf{s}$ | $\mathbf{y}$ | `()` | c`(`v^*`)` | `top`
String	$\mathbf{s}$	:=	x | c
Identifier	x	:=	(`a-z`)(`a-zA-Z0-9_`)*
Constant	c	:=	(`A-Z`)(`a-zA-Z0-9_`)*
Integer	$\mathbf{n}$		HexInteger $\mathbf{h}$
Boolean	$\mathbf{b}$		ByteString $\mathbf{y}$

Type	τ	:=	τ_{atom} | `bytes` | `list` | `packet` | τ_{atom}`[`τ`]`
AtomType	τ_{atom}	:=	`unit` | `int` | `bool` | `string` | c
CompType	$\tau_{\mathbb{C}}$	:=	`ControlPlane` | `DataPlane`
ArrowType	τ_{arrow}	:=	τ_ρ `->` $\tau_\rho{}^+$
PortType	τ_ρ	:=	`_` | τ(`@` τ_ρ)$^?$

DSL Design Rationale The DSL models CubeSat behavior at the level of inter-subsystem interactions. This level of abstraction lies between message formats and implementation details: a description of individual messages alone cannot capture the state changes, control-flow decisions, and subsequent communications that a message may induce, whereas a model of firmware internals would be tied to mission-specific software and hardware. Inter-subsystem communication instead provides a stable, observable boundary at which heterogeneous subsystems coordinate interactions and expose their effects to the rest of the system. The DSL realizes this abstraction by representing subsystems as stateful objects, communication media and topology as resources and interfaces, communication procedures as pipes, and communication control logic as workers. Other subsystem behaviors, including internal processing and protocol encoding and decoding logic, are captured as algorithms. These constructs provide a uniform representation of mission-specific structure and behavior, enabling the systematic generation of telecommands from specified message-sending behaviors and the validation of both message-level and system-level behavior. Furthermore, by supporting abstract values, the DSL allows unresolved data, protocol operations, and behaviors to remain underspecified. Such partial specifications can be incrementally refined as subsystem behaviors and protocols are progressively finalized, enabling specification-driven testing before all implementation details are available.

Specification A specification $\mathcal{S}$ is a sequence of five kinds of building blocks: imports $\mathbb{M}$, resources $\mathbb{R}$, interfaces $\mathbb{I}$, protocols $\mathbb{P}$, and objects $\mathbb{O}$. At runtime, these building blocks are used to construct the initial state of the CubeSat system, which captures the structure of the subsystems and the communication links between them. The behaviors defined within each subsystem are then invoked, enabling the observation of the resulting communication traces among subsystems.

Building Blocks Imports specify the names of files to be included, while the remaining blocks describe distinct aspects of the CubeSat system.

A combination of a resource declaration '`resource` $c : \tau$' and an interface declaration '`interface` x : $c\ \{v_1 : c_1, \cdots, v_n : c_n\}$' defines the communication topology between objects. A resource declaration represents a shared communication medium named c, such as an RF link or a serial communication channel. Each resource is associated with a key type τ, which determines how communication endpoints are identified. Accordingly, an interface named x declares a mapping from endpoint addresses v_i (of type τ) to the connected object names c_i, specifying how multiple objects share access to the same resource.

A protocol '`protocol` $c\ \{a_1 \cdots a_n\}$' defines how messages are encoded and decoded over an interface using algorithms a_i, which resemble function definitions. Each algorithm definition '`algorithm` x_1`(`x_2 : τ_1`)` : $\tau_2\ \{\, e \,\}$' consists of the name x_1, one input x_2 with its type τ_1, the output type τ_2, and the body expression e.

An object models a CubeSat subsystem, specifying its internal behavior. When an object is declared with the `objectC` keyword, it denotes a *controllable object* that initiates transmission and typically corresponds to the CubeSat component under test. An object declaration contains a sequence of internal data d^* to store local states, an optional `init` algorithm to initialize the object's state, a sequence of functional components $\mathbb{C}^*$ to define the object's behaviors, and workers w to specify the execution logic. A data declaration '`data` $x : \tau$' describes a state variable x with its type τ. A component '`component` $c : \tau_{\mathbb{C}}\ \{a_1 \cdots a_n\ p_1 \cdots p_m\}$' has its name c and type $\tau_{\mathbb{C}}$, and defines reusable behavioral units with algorithms a_i and pipes p_j.

Pipes A pipe '`pipe` $x($ `@`$c_1, \cdots, c_n)^?$: $\tau_{\text{arrow}}\ \{\, \mathbf{p} \,\}$' consists of the name x, optional port arguments c_i, the arrow type τ_{arrow} illustrating its behavior, and the pipe body $\mathbf{p}$. It resembles an algorithm; however, unlike an algorithm, a pipe may produce side effects, such as sending messages or modifying data in an object. Within a pipe body, individual steps are chained using `--`, which denotes a sequential execution: each step is evaluated in order, passing intermediate results where appropriate. Each step '$\mathbf{p}_b($ `@`$\rho_1, \cdots, \rho_n)^?$' represents a basic action $\mathbf{p}_b$ bound with communication endpoints ρ_i.

Pipe actions can be classified into three kinds: (1) calling a predefined action $\mathbf{p}_l$ without any arguments, (2) calling a predefined action $\mathbf{p}_f$ with arguments $\mathbf{p}_{c1}, \cdots, \mathbf{p}_{cn}$, or (3) selecting a pipe

body via pattern matching '`matchthis` | e_{p_1} `=>` $\mathbf{p}_1$ $\cdots$ | e_{p_m} `=>` $\mathbf{p}_m$ `end`.' For $\mathbf{p}_l$ actions, three common operations are available, as well as calls to other pipes, algorithms, or built-in functions l. The three common operations are: (1) `sender` to send messages, (2) `receiver` to wait to receive messages, and (3) `ignorer` to terminate the current execution. For $\mathbf{p}_f$ actions, execution is restricted to five common operations: (1) `constant(`e`)` to construct a constant e, (2) `sleeper(`e`)` to suspend the current execution for the number of milliseconds obtained by evaluating e, (3) `syncComm(`$\mathbf{p}_1, \mathbf{p}_2$`)` and (4) `syncCommVal(`$\mathbf{p}_1, \mathbf{p}_2$`)` to synchronously communicate without and with a feedback value, and (5) `looper(`$e, \mathbf{p}$`)` to repeat a pipe $\mathbf{p}$ for a specific number of times e.

Workers Workers '`workers [` $\mathbf{w}_1, \cdots, \mathbf{w}_n$ `]`' define the execution logic that orchestrates components within an object. In each worker definition $\mathbf{w}_i$, the execution is specified as $c.x$, which refers to the pipe x of component c within the same object. The execution logic is either waiting to respond to incoming messages from a specific endpoint '`Listen` $c.x$ `on` x`[`v`]`,' repeatedly invoking the execution '`Trigger` $c.x$ `every` δ `s`,' or invoking the execution once '`Init` $c.x$.'

A `Trigger` delay δ is either an integer literal $\mathbf{n}$ or a parenthesized expression `(`e`)`, measured in seconds. At the beginning of each iteration, the worker obtains its delay, waits for that duration, and executes its pipe. The next iteration begins when the pipe completes, so the delay is measured from the end of the preceding execution. The first execution also follows a wait. For a literal delay, each iteration uses the specified constant. On the other hand, in case of an expression-valued delay `(`e`)`, the worker evaluates e in the current state of its owning object at the beginning of every iteration. The expression may read object data or call an algorithm; any state changes made during its evaluation are retained. It must produce a positive integer number of seconds. State changes during an ongoing wait do not interrupt or shorten that wait; they affect the delay when the expression is next evaluated. For example, the following worker selects its delay using an algorithm: '"`Trigger Sensor.sample every (call Sensor.period_seconds(())) s`'. This would be rejected if the result of the expression '`(call Sensor.period_seconds(()))`' is either non-integer, nonpositive integer, or abstract value. Additionaly, values greater than $\lfloor (2^{31} - 1)/1000 \rfloor$ seconds would be rejected to prevent overflow when converting the delay to a 32-bit millisecond duration.

An `Init` worker can express a one-time startup delay by beginning its pipe with `sleeper($5000)`, for example, to wait five seconds before initialization actions. A sleeper suspends only its current worker; other workers may continue. Unlike `Trigger` delays, sleeper arguments are measured in milliseconds.

Expressions Communication endpoints are represented by port expressions ρ. These include muted ports `_`, port variables c, and ports addressed through an interface x`[`v`]`.

Basic expressions e include values v, identifiers x, parenthesized expressions `(`e`)`, integer binary operations 'e_1 *bop* e_2,' bitwise complement '`~`e,' assignments 'x `=` e_1`;` e_2,' construction of variants 'c`(`$e_1, \cdots, e_n$`)`,' pattern matching, and calls. In a pattern matching expression '`match` e `with` | e_{p_1} `=>` e_1 $\cdots$ | e_{p_n} `=>` e_n `end`,' each expression pattern e_{p_i} is either the wildcard `_`, a symbol x, a value v, or a constructor with subpatterns c`(`$e_{p_1}, \cdots, e_{p_n}$`)`.' A call expression '`call` l`(`$e_1, \cdots, e_n$`)`' invokes a callee, which is either an algorithm, pipe, or function. A callee has a name l, which may be an unqualified identifier x or an identifier qualified by an explicit scope $c.x$, and has arguments e_i.

Integer operators have the following precedence, from highest to lowest. Parentheses override this order. All binary operators associate to the left; the unary complement associates to the right.

Operators	Meaning
`~`	Bitwise complement
`*`, `/`	Multiplication, division
`+`, `-`	Addition, subtraction
`<<`, `>>`, `>>>`	Left, arithmetic right, logical right shift
`&`	Bitwise AND
`^`	Bitwise XOR
`|`	Bitwise OR

These operators take and return 32-bit signed `int` values. The arithmetic right shift `>>` replicates the sign bit, whereas `>>>` fills vacated bits with zeros. Shift distances use only their low five bits, so `1 << 32` evaluates to 1. For example, `0x80000000 >> 31` evaluates to -1, while `0x80000000 >>> 31` evaluates to 1. Bitwise complement affects all 32 bits; `~flags & 0xff` restricts the result to one byte. Bit fields can be extracted and reassembled explicitly. For example, the following expression extracts bits 0–2 and clears the remaining bits of `flags`:

```
selection = flags & 0x03; enabled = (flags >> 2) & 0x01; selection | (enabled << 2)
```

Values Values v include primitive literals such as integers **n**, hexadecimal integers **h**, booleans **b**, strings **s**, and byte strings **y**. Strings are divided into identifiers x and constants c, which start with a lower-case letter and an upper-case letter, respectively. Values also include the unit value `()`, variant values 'c`(`$v_1, \cdots, v_n$`),`' and the abstract top value `top`. This abstraction supports both concrete and abstract representations of CubeSat systems, allowing developers to write incomplete models during early development phases without committing to concrete values or full implementations. Crucially, this enables testing to begin even before firmware is fully implemented or subsystem hardware is available, by allowing unspecified components to be modeled abstractly in the specification.

Types Every expression has a type τ, which may be an atomic type τ_{atom}, a byte sequence `bytes`, a list `list`, a communication payload `packet`, or a parameterized type τ_{atom}`[`τ`]`. Atomic types τ_{atom} include the primitive types `unit`, `int`, `bool`, and `string`, as well as user-defined atomic types denoted by constants c.

The type of a component $\tau_{\mathbb{C}}$ is either `ControlPlane`, governing system behaviors and configurations, or `DataPlane`, processing and transmitting data.

The type of a pipe τ_{arrow} is written as τ_{ρ_1} `->` $(\tau_{\rho_2}$`|`$\cdots$`|`$\tau_{\rho_n})$, where τ_{ρ_1} denotes the type of the source port and $(\tau_{\rho_2}, \cdots, \tau_{\rho_n})$ denote the types of the target ports. Each port type τ_ρ represents either a muted port `_`, an expression type τ, or an expression type annotated with a port type (τ `@`τ_ρ).

2 Semantics

2.1 Domains

$$
\begin{array}{rcll}
\langle \mathcal{D}, \mathcal{B}, \sigma, [k_1, \cdots, k_n] \rangle & \in & \mathit{State} & = \mathit{ProgBase} \times \mathit{Builtin} \times \mathit{Store} \times \mathit{List}[\mathit{PackedKont}] \\
\mathcal{D} & \in & \mathit{ProgBase} & = \mathit{Interfaces} \times \mathit{Protocols} \times \mathit{Objects} \\
\mathcal{I} & \in & \mathit{Interfaces} & = \mathit{Var} \overset{\text{fin}}{\twoheadrightarrow} \mathit{InterfaceInfo} \\
(c, [(v_1, c_1), \cdots, (v_n, c_n)]) & \in & \mathit{InterfaceInfo} & = \mathit{Const} \times \mathit{List}[(\mathit{Val}, \mathit{Const})] \\
\mathcal{P} & \in & \mathit{Protocols} & = \mathit{Const} \overset{\text{fin}}{\twoheadrightarrow} \{\mathbb{P}_1, \cdots, \mathbb{P}_n\} \\
\mathcal{O} & \in & \mathit{Objects} & = \mathit{Const} \overset{\text{fin}}{\twoheadrightarrow} (\mathit{Components} \times \mathit{List}[\mathbf{w}_1, \cdots, \mathbf{w}_n]) \\
\mathcal{C} & \in & \mathit{Components} & = \mathit{Const} \overset{\text{fin}}{\twoheadrightarrow} \{\mathbb{C}_1, \cdots, \mathbb{C}_n\} \\
\mathcal{B} & \in & \mathit{Builtin} & = \mathit{Const} \overset{\text{fin}}{\twoheadrightarrow} (\mathit{Var} \overset{\text{fin}}{\twoheadrightarrow} \mathit{BuiltinFun}) \\
 & & \mathit{BuiltinFun} & = (\mathit{ObjStore} \times \mathit{List}[\mathit{Val}]) \overset{\text{fin}}{\twoheadrightarrow} (\mathit{Val} \times \mathit{PortVal} \times \mathit{ObjStore}) \\
\sigma & \in & \mathit{Store} & = \mathit{ObjStore} \times \mathit{InterfaceStore} \\
\sigma_o & \in & \mathit{ObjStore} & = \mathit{Const} \overset{\text{fin}}{\twoheadrightarrow} (\mathit{ObjData} \times \mathit{ObjMem}) \\
\sigma_o^d & \in & \mathit{ObjData} & = \mathit{Var} \overset{\text{fin}}{\twoheadrightarrow} \mathit{Val} \\
\sigma_o^m & \in & \mathit{ObjMem} & = \mathit{Addr} \overset{\text{fin}}{\twoheadrightarrow} \mathit{List}[\mathit{Val}] \\
\sigma_I & \in & \mathit{InterfaceStore} & = \mathit{Var} \overset{\text{fin}}{\twoheadrightarrow} ((\mathit{Val} \times \mathit{Val}) \overset{\text{fin}}{\twoheadrightarrow} \mathit{List}[\mathit{Time} \times \mathit{Val}]) \\
k & \in & \mathit{PackedKont} & = \mathit{Const} \times \mathit{Kont} \times \mathit{Val} \times \mathit{Kont} \times \mathit{Time} \\
\kappa & \in & \mathit{Kont} & = \{\ \mathsf{period}(e, \kappa),\ \mathsf{sleep}(t, \kappa),\ \mathsf{constant}(v, \kappa), \\
 & & & \quad \mathsf{send}(\rho_I, \kappa),\ \mathsf{receive}(\rho_I, \kappa),\ \mathsf{receivewait}(\rho_I, \kappa), \\
 & & & \quad \mathsf{trigger}(\gamma, \mathbf{p}, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa),\ \mathsf{fin}(\kappa),\ \square\ \} \\
x[v] = \rho_I & \in & \mathit{InterfacePort} & = \mathit{Var} \times \mathit{Val} \\
\gamma & \in & \mathit{BindContext} & = \{\ [\,],\ [\,\mathbb{C}\,],\ [\,\mathbb{P}\,]\} \\
\Delta_{\mathrm{v}} & \in & \mathit{ValEnv} & = \mathit{Var} \overset{\text{fin}}{\twoheadrightarrow} \mathit{Val} \\
\Delta_{\mathrm{p}} & \in & \mathit{PortEnv} & = \mathit{Const} \overset{\text{fin}}{\twoheadrightarrow} \mathit{PortVal} \\
x & \in & \mathit{Var} & = \text{a set of identifiers} \\
c & \in & \mathit{Const} & = \text{a set of constants} \\
v_\rho & \in & \mathit{PortVal} & = \{\ _,\ \rho_I,\ \bot\ \} \\
t & \in & \mathit{Time} & = \mathbb{N}
\end{array}
$$

2.2 Initial state

$\langle \mathcal{D}, \mathcal{B}, \sigma, [k_1 \cdots k_n] \rangle$

$$
\begin{array}{rcl}
\mathcal{D} & = & \langle \mathcal{I}, \mathcal{P}, \mathcal{O} \rangle \\
\mathcal{I} & = & [x_j \mapsto (c_j, [(v_{j_1}, c_{j_1}), \cdots, (v_{j_{m_j}}, c_{j_{m_j}})])] \\
 & & \text{for all } \mathbb{I}_j = \texttt{interface}\ x_j\ : c_j\ \{\ v_{j_1} : c_{j_1}\ \cdots\ v_{j_{m_j}} : c_{j_{m_j}}\ \} \in \mathcal{S} \\
\mathcal{P} & = & [c_j \mapsto \mathbb{P}_j] \text{ for all } \mathbb{P}_j = \texttt{protocol}\ c_j\ \{\ a_{j_1} \cdots a_{j_{n_j}}\ \} \in \mathcal{S} \\
\mathcal{O} & = & [c_j \mapsto (\mathcal{C}_j, [\mathbf{w}_{j_1}, \cdots, \mathbf{w}_{j_{n_j}}])] \\
 & & \text{for all } \mathbb{O}_j = (\texttt{object} \mid \texttt{objectC})\ c_j\ \{ \cdots \mathbb{C}_{j_1} \cdots \mathbb{C}_{j_{m_j}}\ \texttt{workers [}\ \mathbf{w}_{j_1} \cdots \mathbf{w}_{j_{n_j}}\ \texttt{]}\ \} \in \mathcal{S} \\
 & & \text{where } \mathcal{C}_j = [c_{j_h} \mapsto \mathbb{C}_{j_h}] \text{ for all } \mathbb{C}_{j_h} = (\texttt{component}\ c_{j_h}\ \cdots) \\
\mathcal{B} & & \text{a set of user-defined packages that define their functions} \\
\sigma & = & (\sigma_o^n, []) \text{ where } n \text{ is the number of objects in } \mathcal{S} \\
\sigma_o^j & = & \begin{cases} \{\} & \text{if } j = 0 \\ \mathcal{D}, \mathcal{B}, c_j, \sigma_o^{j-1}, [], [], [], \square \vdash a_{\text{init}_j} \mid\mid (), \square \Downarrow (), \square, {\sigma_o}^j & \text{if } j \in [1,n],\ \exists a_{\text{init}_j} \\ \sigma_o^{j-1} & \text{if } j \in [1,n],\ \nexists a_{\text{init}_j} \end{cases} \\
 & & \text{where } j \in [1,n],\ \{\mathbb{O}_1, \cdots, \mathbb{O}_n\} \text{ is the set of all objects in } \mathcal{S}, \text{ and} \\
 & & \mathbb{O}_j = (\texttt{object} \mid \texttt{objectC})\ c_j\ \{\ d_{j_1} \cdots d_{j_{l_j}}\ a^?_{\text{init}}\ \cdots\ \} \\
[k_1 \cdots k_n] & = & \bigcup_{\mathbb{O}_j \in \mathcal{S}} \left\{ k_{j_1}, \cdots, k_{j_{n_j}} \;\middle|\; \begin{array}{l} \mathbb{O}_j = (\texttt{object} \mid \texttt{objectC})\ c_j\ \{ \cdots\ \texttt{workers [}\ \mathbf{w}_{j_1} \cdots \mathbf{w}_{j_{n_j}}\ \texttt{]}\ \}, \\ \kappa_{j_h} = \mathsf{initWorker}(\mathcal{D}, c_j, \mathbf{w}_{j_h}),\ k_{j_h} = \langle c_j, \kappa_{j_h}, (), \kappa_{j_h}, 0 \rangle \end{array} \right\} \\
 & & \text{where } n = \sum_{\mathbb{O}_j \in \mathcal{S}} n_j
\end{array}
$$

2.3 State transition rule

$$
\langle \mathcal{D}, \mathcal{B}, \sigma, [k_1 \cdots k_n] \rangle \Rightarrow \bigcup_{i \in [1,n]} \left\{ \langle \mathcal{D}, \mathcal{B}, \sigma_{i\,j}, K_{i\,j} \rangle \;\middle|\; \begin{array}{l} \mathcal{D}, \mathcal{B} \vdash [\sigma, k_i] \rightsquigarrow \{[\sigma_{i\,1}, k_{i\,1}], \cdots, [\sigma_{i\,m}, k_{i\,m}]\}, \\ K_{i\,j} = [k_1 \cdots k_{i-1}, k_{i\,j}, k_{i+1} \cdots k_n] \end{array} \right\}
$$

2.4 Evaluation rules: Worker sequence (continuation)

$$\frac{(\sigma_o, \sigma_I) = \sigma \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, [], [], [], \kappa_{\text{init}} \vdash e \mid\mid (), \square \Downarrow n, \square, \sigma_o' \qquad n \in \mathbb{Z} \quad 0 < n \leq \lfloor (2^{31}-1)/1000 \rfloor \quad \sigma' = (\sigma_o', \sigma_I)}{\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v, \mathsf{period}(e, \kappa), t \rangle] \rightsquigarrow \{[\sigma', \langle \mathbf{s}_o, \kappa_{\text{init}}, (), \mathsf{sleep}(1000n, \kappa), t \rangle]\}}$$

$$\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \mathsf{sleep}(t_2, \kappa), t_1 \rangle] \rightsquigarrow \{[\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, (), \kappa, t_1 + t_2 \rangle]\}$$

$$\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \mathsf{constant}(v_2, \kappa), t_1 \rangle] \rightsquigarrow \{[\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_2, \kappa, t_1 \rangle]\}$$

$$\frac{x_I[v_I] = \rho_I \quad \langle \mathcal{I}, \mathcal{P}, \mathcal{O} \rangle = \mathcal{D} \quad (c, [(v_1, c_1), \cdots, (v_n, c_n)]) = \mathcal{I}(x_I) \quad j \in [1, n] \quad c_j = \mathbf{s}_o \quad (\sigma_o, \sigma_I) = \sigma \quad \mathsf{lst} = (t_0, v_0) :: \sigma_I(x_I)((v_j, v_I)) \quad \sigma_I' = \sigma_I[x_I \mapsto \sigma_I(x_I)((v_j, v_I) \mapsto \mathsf{lst})] \quad \sigma' = (\sigma_o, \sigma_I')}{\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_0, \mathsf{send}(\rho_I, \kappa), t_0 \rangle] \rightsquigarrow \{[\sigma', \langle \mathbf{s}_o, \kappa_{\text{init}}, (), \kappa, t_0 \rangle]\}}$$

$$\frac{x_I[v_I] = \rho_I \quad \langle \mathcal{I}, \mathcal{P}, \mathcal{O} \rangle = \mathcal{D} \quad (c, [(v_1, c_1), \cdots, (v_n, c_n)]) = \mathcal{I}(x_I) \quad j \in [1, n] \quad c_j = \mathbf{s}_o \quad (\sigma_o, \sigma_I) = \sigma \quad (t_1, v_1) :: \mathsf{tl} = \sigma_I(x_I)((v_I, v_j)) \quad \sigma_I' = \sigma_I[x_I \mapsto \sigma_I(x_I)((v_I, v_j) \mapsto \mathsf{tl})] \quad \sigma' = (\sigma_o, \sigma_I')}{\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_0, \mathsf{receive}(\rho_I, \kappa), t_0 \rangle] \rightsquigarrow \{[\sigma', \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \kappa, \mathsf{max}(t_0, t_1) \rangle]\}}$$

$$\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \mathsf{receivewait}(\rho_I, \kappa), t_1 \rangle] \rightsquigarrow \{[\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \mathsf{receive}(\rho_I, \kappa), t_1 \rangle]\}$$

$$\frac{\kappa \neq \square}{\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \mathsf{fin}(\kappa), t_1 \rangle] \rightsquigarrow \{[\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, (), \kappa, t_1 \rangle]\}}$$

$$\frac{(\sigma_o, \sigma_I) = \sigma \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa_{\text{init}} \vdash \mathbf{p} \mid\mid v_1, \kappa \Downarrow v_2, \kappa', \sigma_o' \quad \sigma' = (\sigma_o', \sigma_I)}{\mathcal{D}, \mathcal{B} \vdash [\sigma, \langle \mathbf{s}_o, \kappa_{\text{init}}, v_1, \mathsf{trigger}(\gamma, \mathbf{p}, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa), t_1 \rangle] \rightsquigarrow \{[\sigma', \langle \mathbf{s}_o, \kappa_{\text{init}}, v_2, \kappa', t_1 \rangle]\}}$$

- Time t is measured in milliseconds.
- When κ is $\mathsf{period}(e, \kappa')$, it is evaluated into a `Trigger` delay δ in the owning object's current store, with empty binding, variable, and port contexts. It preserves any store changes and schedules a sleep before κ'. The resulting delay must be a positive integer number of seconds whose millisecond representation fits in a 32-bit signed integer; other results cause an execution error.

2.5 Evaluation rules: Algorithms

$$\frac{(\sigma_o^d, \sigma_o^m) = \sigma_o(\mathbf{s}_o) \quad x_{\text{arg}} \notin \sigma_o^d \qquad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}[x_{\text{arg}} \mapsto v], \Delta_{\text{p}}, \kappa_{\text{init}} \vdash e \mid\mid v, \square \Downarrow v', \square, \sigma_o'}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa_{\text{init}} \vdash \texttt{algorithm}\ x_{\text{name}}\ (x_{\text{arg}} : \tau_1)\ \tau_2\ \{\ e\ \} \mid\mid v, \square \Downarrow v', \square, \sigma_o'}$$

2.6 Evaluation rules: Worker actions (pipe expressions)

2.6.1 Composed pipe combinators

$$\frac{\kappa' = \mathsf{trigger}(\gamma, \mathbf{p}_1, \Delta_{\text{v}}, \Delta_{\text{p}}, \mathsf{trigger}(\gamma, \mathbf{p}_2, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa))}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa_{\text{init}} \vdash \mathbf{p}_1 \texttt{ -- } \mathbf{p}_2 \mid\mid v, \kappa \Downarrow v, \kappa', \sigma_o}$$

2.6.2 Ported pipe combinators

$$\frac{\begin{gathered}\forall j \in [1, n].\{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa_{\text{init}} \vdash \rho_j \mid\mid v, \square \Downarrow v_{\rho_j}, \square, \sigma_o, v_{\rho_j} \neq \bot\} \\ [c_1, \cdots, c_n] = \begin{cases} \mathsf{resolvePortparm}(\mathbf{p}_l, \gamma) & \text{if } \mathbf{p}_b = \mathbf{p}_l \\ \mathsf{resolvePortparm}(\mathbf{p}_f, \gamma) & \text{if } \mathbf{p}_b = \mathbf{p}_f(\mathbf{p}_{c1}, \cdots, \mathbf{p}_{cn}) \end{cases} \\ \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}, \Delta_{\text{p}}[c_1 \mapsto v_{\rho_1}, \cdots, c_n \mapsto v_{\rho_n}], \kappa_{\text{init}} \vdash \mathbf{p}_b \mid\mid v, \kappa \Downarrow v', \kappa', \sigma_o'\end{gathered}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\text{v}}, \Delta_{\text{p}}, \kappa_{\text{init}} \vdash \mathbf{p}_b \texttt{ @ } \rho_1, \cdots, \rho_n \mid\mid v, \kappa \Downarrow v', \kappa', \sigma_o'}$$

2.6.3 Non-ported pipe combinators w/ qualified name

$$\frac{\mathbf{p}_l \in \{\texttt{sender}, \texttt{receiver}, \texttt{ignorer}\} \quad (v', \kappa') = \begin{cases} (v, \mathsf{send}(\Delta_{\mathrm{p}}(\texttt{To}), \kappa)) & \text{if } \mathbf{p}_l = \texttt{sender} \\ (v, \kappa) & \text{if } \mathbf{p}_l = \texttt{receiver} \\ ((), \mathsf{fin}(\kappa_{\mathrm{init}})) & \text{if } \mathbf{p}_l = \texttt{ignorer} \end{cases}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash \mathbf{p}_l \ || \ v, \kappa \ \Downarrow \ v', \kappa', \sigma_o}$$

$$\frac{\begin{array}{c} x \notin \{\texttt{sender}, \texttt{receiver}, \texttt{ignorer}\} \quad [\ \texttt{protocol}\ c_{\mathrm{pname}}\ \{\ a_1, \cdots, a_n\ \}\ ] = \gamma \\ \forall j \in [1, n].\{\texttt{algorithm}\ x_{\mathrm{aname}_j}\ (x_{\mathrm{arg}_j} : \tau_{\mathrm{arg}_j})\ \tau_j\ \{\ e_j\ \} = a_j\} \quad x = x_{\mathrm{aname}_j} \\ \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash a_j \ || \ v, \square \ \Downarrow \ v', \square, {\sigma_o}' \end{array}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash x \ || \ v, \kappa \ \Downarrow \ v', \kappa, {\sigma_o}'}$$

$$\frac{\begin{array}{c} x \notin \{\texttt{sender}, \texttt{receiver}, \texttt{ignorer}\} \quad [\ \texttt{component}\ c_{\mathrm{cname}}\ : \tau_{\mathbb{C}}\ \{\ a_1, \cdots, a_m, p_1, \cdots, p_n\ \}\ ] = \gamma \\ \forall j \in [1, n].\{\texttt{pipe}\ x_{\mathrm{pname}_j}\ \texttt{@}\ c_{j_1}, \cdots, c_{j_h}\ : \tau_{\mathrm{arrow}j}\ \{\ \mathbf{p}_j\ \} = p_j\} \qquad x = x_{\mathrm{pname}_j} \end{array}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash x \ || \ v, \kappa \ \Downarrow \ v, \mathsf{trigger}(\gamma, \mathbf{p}_j, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa), \sigma_o}$$

$$\frac{\mathsf{definedBuiltin}(\mathcal{B}, c, x) \quad (v', v_\rho, \sigma_o') = \mathsf{executeBuiltin}(\mathcal{B}, c, x, [v], \sigma_o) \quad \kappa' = \begin{cases} \mathsf{fin}(\kappa_{\mathrm{init}}) & \text{if } v_\rho = _ \\ \mathsf{send}(\rho_I, \kappa) & \text{if } v_\rho = \rho_I \\ \kappa & \text{if } v_\rho = \bot \end{cases}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash c.x \ || \ v, \kappa \ \Downarrow \ v', \kappa', {\sigma_o}'}$$

$$\frac{\begin{array}{c} \neg\mathsf{definedBuiltin}(\mathcal{B}, c, x) \quad \langle \mathcal{I}, \mathcal{P}, \mathcal{O} \rangle = \mathcal{D} \quad \texttt{protocol}\ c\ \{\ a_1, \cdots, a_n\ \} = \mathcal{P}(c) \\ \forall j \in [1, n].\{\texttt{algorithm}\ x_{\mathrm{aname}_j}\ (x_{\mathrm{arg}_j} : \tau_{\mathrm{arg}_j})\ \tau_j\ \{\ e_j\ \} = a_j\} \quad x = x_{\mathrm{aname}_j} \\ \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, [\ \mathcal{P}(c)\ ], \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash a_j \ || \ v, \square \ \Downarrow \ v', \square, {\sigma_o}' \end{array}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash c.x \ || \ v, \kappa \ \Downarrow \ v', \kappa, {\sigma_o}'}$$

$$\frac{\begin{array}{c} \neg\mathsf{definedBuiltin}(\mathcal{B}, c, x) \quad \langle \mathcal{I}, \mathcal{P}, \mathcal{O} \rangle = \mathcal{D} \quad (\mathcal{C}, [\mathbf{w}_1, \cdots, \mathbf{w}_g]) = \mathcal{O}(\mathbf{s}_o) \\ \texttt{component}\ c_{\mathrm{cname}}\ : \tau_{\mathbb{C}}\ \{\ a_1, \cdots, a_m, p_1, \cdots, p_n\ \} = \mathcal{C}(c) \\ \forall j \in [1, n].\{\texttt{pipe}\ x_{\mathrm{pname}_j}\ \texttt{@}\ c_{j_1}, \cdots, c_{j_h}\ : \tau_{\mathrm{arrow}j}\ \{\ \mathbf{p}_j\ \} = p_j\} \quad x = x_{\mathrm{pname}_j} \end{array}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa_{\mathrm{init}} \ \vdash c.x \ || \ v, \kappa \ \Downarrow \ v, \mathsf{trigger}([\ \mathcal{C}(c)\ ], \mathbf{p}_j, \Delta_{\mathrm{v}}, \Delta_{\mathrm{p}}, \kappa), \sigma_o}$$

2.6.4 Non-ported pipe combinators w/ application

$$\frac{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash e \mid\mid v,\kappa \Downarrow v',\square,{\sigma_o}'}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{constant(\$}e\texttt{)} \mid\mid v,\kappa \Downarrow v,\mathsf{constant}(v',\kappa),{\sigma_o}'}$$

$$\frac{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash e \mid\mid v,\kappa \Downarrow n,\square,{\sigma_o}'}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{sleeper(\$}e\texttt{)} \mid\mid v,\kappa \Downarrow v,\mathsf{sleep}(n,\kappa),{\sigma_o}'}$$

$$\frac{\kappa' = \mathsf{trigger}(\gamma,\mathbf{p}_1,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_1) \qquad \kappa_1 = \mathsf{receivewait}(\Delta_\mathrm{p}(\texttt{From}),\kappa_2) \quad \kappa_2 = \mathsf{trigger}(\gamma,\mathbf{p}_2,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa)}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{syncComm(}\mathbf{p}_1,\mathbf{p}_2\texttt{)} \mid\mid v,\kappa \Downarrow v,\kappa',\sigma_o}$$

$$\frac{\begin{array}{c}\kappa' = \mathsf{trigger}(\gamma,\mathbf{p}_1,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_1) \qquad \kappa_1 = \mathsf{receivewait}(\Delta_\mathrm{p}(\texttt{From}),\kappa_2) \quad \kappa_2 = \mathsf{trigger}(\gamma,\mathbf{p}_2,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_3)\\ \kappa_3 = \mathsf{constant}(v,\kappa_4) \quad \kappa_4 = \mathsf{trigger}(\gamma,\mathbf{p}_1,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_5)\\ \kappa_5 = \mathsf{receivewait}(\Delta_\mathrm{p}(\texttt{From}),\kappa_6) \quad \kappa_6 = \mathsf{trigger}(\gamma,\mathbf{p}_2,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa)\end{array}}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{syncCommVal(}\mathbf{p}_1,\mathbf{p}_2\texttt{)} \mid\mid v,\kappa \Downarrow v,\kappa',\sigma_o}$$

$$\frac{\begin{array}{c}\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash e \mid\mid v,\kappa \Downarrow n,\square,{\sigma_o}'\\ \kappa' = \mathsf{trigger}(\gamma,\mathbf{p},\Delta_\mathrm{v},\Delta_\mathrm{p},\mathsf{trigger}(\gamma,\mathbf{p},\Delta_\mathrm{v},\Delta_\mathrm{p},\mathsf{trigger}\cdots\mathsf{trigger}(\gamma,\mathbf{p},\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa)))\\ \text{(n times nested } \mathsf{trigger} \text{ with } \mathbf{p}\text{)}\end{array}}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{looper(\$}e\texttt{,}\mathbf{p}\texttt{)} \mid\mid v,\kappa \Downarrow v,\kappa',{\sigma_o}'}$$

2.6.5 Non-ported pipe combinators w/ pattern matching

$$\frac{\begin{array}{c}(\Delta'_\mathrm{v},\mathbf{p}_j) = \mathsf{matchPattern}(v,[(e_{p_1},\mathbf{p}_1),\cdots,(e_{p_n},\mathbf{p}_n)]) \quad \Delta'_\mathrm{v} \neq \bot\\ \mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v}\cup\Delta'_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \mathbf{p}_j \mid\mid v,\kappa \Downarrow v',\kappa',\sigma_o'\end{array}}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{matchthis |}e_{p_1}\texttt{ => }\mathbf{p}_1\ \cdots\ \texttt{|}e_{p_n}\texttt{ => }\mathbf{p}_n\ \texttt{end} \mid\mid v,\kappa \Downarrow v',\kappa',\sigma_o'}$$

2.7 Evaluation rules: Expressions

2.7.1 Port expressions

$$\frac{v_\rho = \begin{cases}\rho & \text{if } \rho = _ \mid \rho_I\\ \Delta_\mathrm{p}(c) & \text{if } \rho = c\end{cases}}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \rho \mid\mid v,\square \Downarrow v_\rho,\square,\sigma_o}$$

2.7.2 Basic expressions

$$\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash v \mid\mid v',\kappa \Downarrow v,\square,\sigma_o$$

$$\frac{(\sigma_o^d,\sigma_o^m) = \sigma_o(\mathbf{s}_o) \quad v' = \begin{cases}\sigma_o^d(x) & \text{if } x \in \sigma_o^d\\ \Delta_\mathrm{v}(x) & \text{if } x \in \Delta_\mathrm{v}\end{cases}}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash x \mid\mid v,\kappa \Downarrow v',\square,\sigma_o}$$

$$\frac{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash e \mid\mid v,\square \Downarrow v',\square,{\sigma_o}'}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{(}e\texttt{)} \mid\mid v,\kappa \Downarrow v',\square,{\sigma_o}'}$$

$$\frac{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash e \mid\mid v,\square \Downarrow v_1,\square,{\sigma_o}'}{\mathcal{D},\mathcal{B},\mathbf{s}_o,\sigma_o,\gamma,\Delta_\mathrm{v},\Delta_\mathrm{p},\kappa_\mathrm{init} \vdash \texttt{~}e \mid\mid v,\kappa \Downarrow \mathsf{complement}(v_1),\square,{\sigma_o}'}$$

$$\frac{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \mid\mid v, \square \Downarrow v_1, \square, {\sigma_o}' \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}', \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_2 \mid\mid v, \square \Downarrow v_2, \square, {\sigma_o}''}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \ bop \ e_2 \mid\mid v, \kappa \Downarrow \mathsf{binary}_{bop}(v_1, v_2), \square, {\sigma_o}''}$$

$$\frac{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \mid\mid v, \square \Downarrow v_1, \square, {\sigma_o}' \quad (\sigma_o^d, \sigma_o^m) = \sigma_o'(\mathbf{s}_o) \quad ({\sigma_o^d}', {\Delta_\mathrm{v}}') = \begin{cases} (\sigma_o^d[x \mapsto v_1], \Delta_\mathrm{v}) & \text{if } x \in \sigma_o^d \\ (\sigma_o^d, \Delta_\mathrm{v}[x \mapsto v_1]) & \text{otherwise} \end{cases} \quad \sigma_o'' = ({\sigma_o^d}', \sigma_o^m) \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}'', \gamma, {\Delta_\mathrm{v}}', \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_2 \mid\mid v, \square \Downarrow v_2, \square, {\sigma_o}'''}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash x \texttt{ = } e_1 \ ; \ e_2 \mid\mid v, \kappa \Downarrow v_2, \square, {\sigma_o}'''}$$

$$\frac{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \mid\mid v, \square \Downarrow v_1, \square, {\sigma_o}^1 \quad \forall i \in [1, n-1].\{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}^i, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_{i+1} \mid\mid v, \square \Downarrow v_{i+1}, \square, {\sigma_o}^{i+1}\}}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash c(e_1, \ \cdots, \ e_n) \mid\mid v, \kappa \Downarrow c(v_1, \ \cdots, \ v_n), \square, {\sigma_o}^n}$$

$$\frac{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e \mid\mid v, \square \Downarrow v', \square, {\sigma_o}' \quad (\Delta_\mathrm{v}', e_j) = \mathsf{matchPattern}(v, [(e_{p_1}, e_1), \cdots, (e_{p_n}, e_n)]) \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v} \cup \Delta_\mathrm{v}', \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_j \mid\mid v, \square \Downarrow v'', \square, {\sigma_o}''}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash \texttt{match } e \texttt{ with | } e_{p_1} \texttt{ => } e_1 \ \cdots \ \texttt{| } e_{p_n} \texttt{ => } e_n \texttt{ end} \mid\mid v, \kappa \Downarrow v'', \square, {\sigma_o}''}$$

$$\frac{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \mid\mid v, \square \Downarrow v_1, \square, {\sigma_o}^1 \quad \forall i \in [1, n-1].\{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}^i, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_{i+1} \mid\mid v, \square \Downarrow v_{i+1}, \square, {\sigma_o}^{i+1}\} \quad [\ \texttt{protocol } c_\mathrm{pname} \ \{ \ a_1, \cdots, a_h \ \} \ ] = \gamma \quad \forall j \in [1, h].\{\texttt{algorithm } x_{\mathrm{aname}_j} \ (x_{\mathrm{arg}_j} : \tau_{\mathrm{arg}_j}) \ \tau_j \ \{ \ e_j \ \} = a_j\} \quad x = x_{\mathrm{aname}_j} \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}^n, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash a_j \mid\mid v, \square \Downarrow v', \square, {\sigma_o}'}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash \texttt{call } x(e_1, \ \cdots, \ e_n) \mid\mid v, \kappa \Downarrow v', \square, {\sigma_o}'}$$

$$\frac{\mathsf{definedBuiltin}(\mathcal{B}, c, x) \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \mid\mid v, \square \Downarrow v_1, \square, {\sigma_o}^1 \quad \forall i \in [1, n-1].\{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}^i, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_{i+1} \mid\mid v, \square \Downarrow v_{i+1}, \square, {\sigma_o}^{i+1}\} \quad (v', v_\rho, {\sigma_o}') = \mathsf{executeBuiltin}(\mathcal{B}, c, x, [v_1, \cdots, v_n], \sigma_o^n)}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash \texttt{call } c.x(e_1, \ \cdots, \ e_n) \mid\mid v, \kappa \Downarrow v', \square, {\sigma_o}'}$$

$$\frac{\neg\mathsf{definedBuiltin}(\mathcal{B}, c, x) \quad \langle \mathcal{I}, \mathcal{P}, \mathcal{O} \rangle = \mathcal{D} \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_1 \mid\mid v, \square \Downarrow v_1, \square, {\sigma_o}^1 \quad \forall i \in [1, n-1].\{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}^i, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash e_{i+1} \mid\mid v, \square \Downarrow v_{i+1}, \square, {\sigma_o}^{i+1}\} \quad \texttt{protocol } c \ \{ \ a_1, \cdots, a_h \ \} = \mathcal{P}(c) \quad \forall j \in [1, h].\{\texttt{algorithm } x_{\mathrm{aname}_j} \ (x_{\mathrm{arg}_j} : \tau_{\mathrm{arg}_j}) \ \tau_j \ \{ \ e_j \ \} = a_j\} \quad x = x_{\mathrm{aname}_j} \quad \mathcal{D}, \mathcal{B}, \mathbf{s}_o, {\sigma_o}^n, [\ \mathcal{P}(c) \ ], \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash a_j \mid\mid v, \square \Downarrow v', \square, {\sigma_o}'}{\mathcal{D}, \mathcal{B}, \mathbf{s}_o, \sigma_o, \gamma, \Delta_\mathrm{v}, \Delta_\mathrm{p}, \kappa_\mathrm{init} \vdash \texttt{call } c.x(e_1, \ \cdots, \ e_n) \mid\mid v, \kappa \Downarrow v', \square, {\sigma_o}'}$$

2.8 Auxiliary functions

- $\mathsf{initWorker}(\mathcal{D}, \mathbf{s}_o, \mathbf{w})$

$$= \begin{cases} \mathsf{receivewait}(\rho_I, \mathsf{trigger}([\ \mathcal{C}(c)\ ], \mathbf{p}_j, [], [], \mathsf{fin}(\kappa))) & \text{if } \mathbf{w} = \texttt{Listen } c.x \texttt{ on } \rho_I \\ \mathsf{sleep}(\mathbf{n} * 1000, \mathsf{trigger}([\ \mathcal{C}(c)\ ], \mathbf{p}_j, [], [], \mathsf{fin}(\kappa))) & \text{if } \mathbf{w} = \texttt{Trigger } c.x \texttt{ every } \mathbf{n}\ \mathbf{s} \\ \mathsf{period}(e, \mathsf{trigger}([\ \mathcal{C}(c)\ ], \mathbf{p}_j, [], [], \mathsf{fin}(\kappa))) & \text{if } \mathbf{w} = \texttt{Trigger } c.x \texttt{ every (}e\texttt{) s} \\ \mathsf{trigger}([\ \mathcal{C}(c)\ ], \mathbf{p}_j, [], [], \square) & \text{if } \mathbf{w} = \texttt{Init } c.x \end{cases}$$

where $\mathcal{D} = \langle _, _, \mathcal{O} \rangle,\ (\mathcal{C}, _) = \mathcal{O}(\mathbf{s}_o),\ \mathcal{C}(c) = \texttt{component } c_{\text{cname}} \cdots \{ \cdots\ p_1, \cdots, p_n\ \},$
$\exists j \in [1, n].\{\texttt{pipe } x_{\text{pname}_j} \cdots \{\ \mathbf{p}_j\ \} = p_j,\ x = x_{\text{pname}_j}\},\ \kappa = \mathsf{initWorker}(\mathcal{D}, \mathbf{s}_o, \mathbf{w})$

For an expression-valued Trigger delay, κ begins with $\mathsf{period}(e, \cdots)$ and retains the expression itself. When the pipe completes, $\mathsf{fin}(\kappa)$ resumes this continuation, which reevaluates the expression in the current store. A state change made by another worker does not replace a pending sleep continuation or interrupt the current pipe execution.

- $\mathsf{resolvePortparm}(l,\ \gamma)$

$$= \begin{cases} [\texttt{To}] & \text{if } l = \texttt{sender} \\ [\texttt{From}] & \text{if } l \in \{\texttt{receiver, syncComm, syncCommVal}\} \\ [c_{j_1}, \cdots, c_{j_h}] & \text{if } l = x,\ l \notin \{\texttt{sender, receiver, syncComm, syncCommVal}\} \\ & \quad \gamma = [\ \texttt{component } c_{\text{cname}} : \tau_{\mathbb{C}}\{\ a_1, \cdots, a_m, p_1, \cdots, p_n\ \}\ ], \text{ and} \\ & \quad \exists j \in [1, n] \text{ s.t. } \{\ p_j = \texttt{pipe } x_{\text{pname}_j}\ \texttt{@}\ c_{j_1}, \cdots, c_{j_h}\ : \tau_{\text{arrow}j}\ \{\ \mathbf{p}_j\ \}, \\ & \qquad\qquad x = x_{\text{pname}_j}\ \} \\ [] & \text{otherwise} \end{cases}$$

- $\mathsf{definedBuiltin}(\mathcal{B}, c, x) = (c \in \mathit{Domain}(\mathcal{B})) \wedge (x \in \mathit{Domain}(\mathcal{B}(c)))$

- $\mathsf{executeBuiltin}(\mathcal{B}, \mathbf{s}_{\text{pkg}}, \mathbf{s}_{\text{f}}, [v_1, \cdots, v_n], \sigma_o) = \mathcal{B}(\mathbf{s}_{\text{pkg}})(\mathbf{s}_{\text{f}})((\sigma_o, [v_1, \cdots, v_n]))$

- $\mathsf{matchPattern}(v, \mathsf{lst})$

$$= \begin{cases} (\Delta, \mathtt{E}_1) & \text{if } \Delta \neq \bot \\ \mathsf{matchPattern}(v, [({e_p}^2, \mathtt{E}_2), \cdots, ({e_p}^n, \mathtt{E}_n)]) & \text{if } \mathsf{len}(\mathsf{lst}) > 1 \text{ and } \Delta = \bot \\ \bot & \text{if } \mathsf{len}(\mathsf{lst}) = 1 \text{ and } \Delta = \bot \end{cases}$$

where $\mathsf{lst} = [({e_p}^1, \mathtt{E}_1), \cdots, ({e_p}^n, \mathtt{E}_n)]$ and $\Delta = \mathsf{matchPattern}(v, {e_p}^1)$

- $\mathsf{matchPattern}(v, e_p)$

$$= \begin{cases} [] & \text{if } e_p = _ \\ [] & \text{if } e_p = \mathbf{h} \mid \mathbf{n} \mid \mathbf{b} \mid \mathbf{s} \mid \mathbf{y} \mid \texttt{()} \text{ and } v = e_p|\texttt{top} \\ [x \mapsto v] & \text{if } e_p = x \\ \bigcup\limits_{j \in [1,n]} \{\Delta_{\text{v}}^j \mid \Delta_{\text{v}}^j \neq \bot\} & \text{if } e_p = c({e_p}^1, \cdots, {e_p}^n),\ v = c(v_1, \cdots, v_n)|\texttt{top} \text{ and} \\ & \quad \Delta_{\text{v}}^j = \mathsf{matchPattern}(v_j, {e_p}^j) \\ \bot & \text{otherwise} \end{cases}$$

- $\mathsf{complement}(v)$

$$= \begin{cases} \texttt{top} & \text{if } v = \texttt{top} \\ -n - 1 & \text{if } v = n \in \mathbb{Z}_{32} \end{cases}$$

- $\mathsf{binary}_{bop}(v_1, v_2)$

$$= \begin{cases} \texttt{top} & \text{if } v_1 = \texttt{top} \text{ or } v_2 = \texttt{top} \\ [\![bop]\!](v_1, v_2) & \text{if } v_1, v_2 \in \mathbb{Z}_{32} \text{ and } (bop \neq \texttt{/} \text{ or } v_2 \neq 0) \end{cases}$$

where $[\![bop]\!]$ is defined as:

$$
\begin{aligned}
[\![\texttt{+}]\!](n_1, n_2) &= (n_1 + n_2) \mod 32 \\
[\![\texttt{-}]\!](n_1, n_2) &= (n_1 - n_2) \mod 32 \\
[\![\texttt{*}]\!](n_1, n_2) &= (n_1 \times n_2) \mod 32 \\
[\![\texttt{/}]\!](n_1, n_2) &= \mathsf{trunc}(n_1 / n_2) \\
[\![\texttt{\&}]\!](n_1, n_2) &= n_1 \,\&_{32}\, n_2 \\
[\![\texttt{|}]\!](n_1, n_2) &= n_1 \,|_{32}\, n_2 \\
[\![\texttt{\^{}}]\!](n_1, n_2) &= n_1 \,\oplus_{32}\, n_2 \\
[\![\texttt{<<}]\!](n_1, n_2) &= (n_1 \ll (n_2 \,\&\, 31)) \,\&\, \texttt{0xFFFFFFFF} \\
[\![\texttt{>>}]\!](n_1, n_2) &= n_1 \gg_s (n_2 \,\&\, 31) \\
[\![\texttt{>>>}]\!](n_1, n_2) &= n_1 \gg_u (n_2 \,\&\, 31)
\end{aligned}
$$

where $\mod 32$ denotes truncation to the signed 32-bit range, $\gg_s$ arithmetic (sign-extending) right shift, and $\gg_u$ logical (zero-extending) right shift. The result type is `int` in all cases.